11. *Header Linked List*

–Traverse In Order

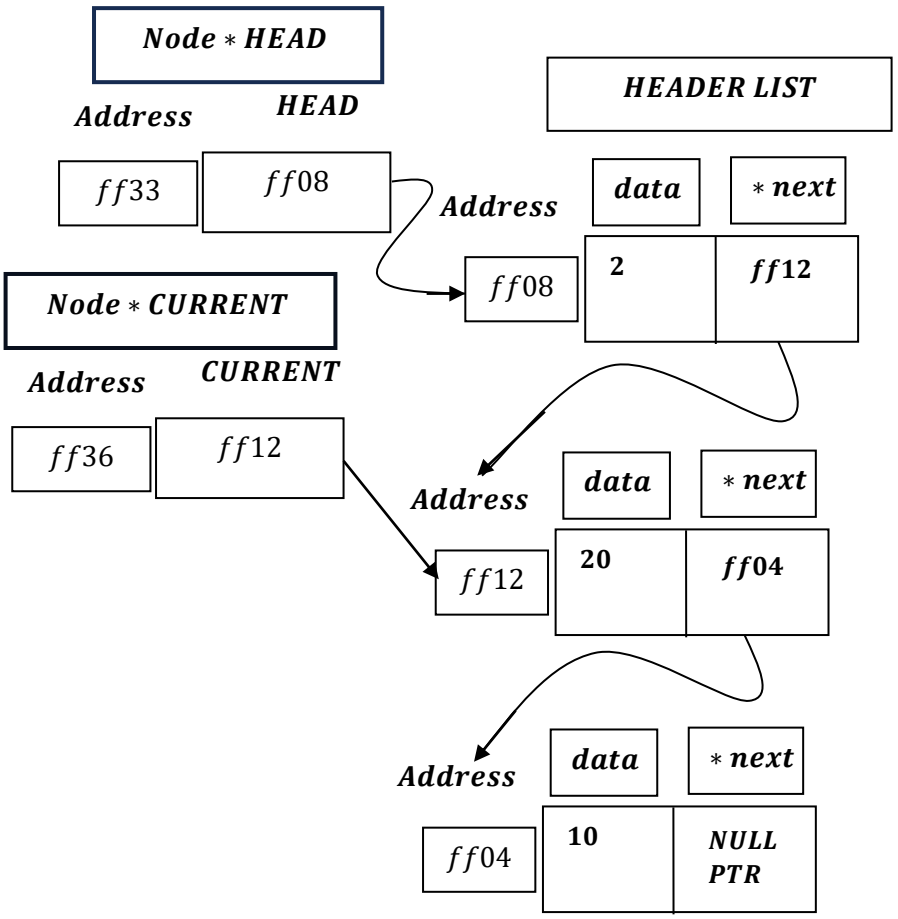
Program:

```
void destroyList()
{
    Node *current = head->next; // Start after header
    Node *nextNode;

    while (current != nullptr)
    {
        nextNode = current->next;
        free(current);
        current = nextNode;
    }

    // Reset header
    head->next = nullptr;
    head->data = 0;

    cout << "List cleared (Header node preserved)." << endl;
}
…
        case 9:
            destroyList();
            break;
```

Program Analysis

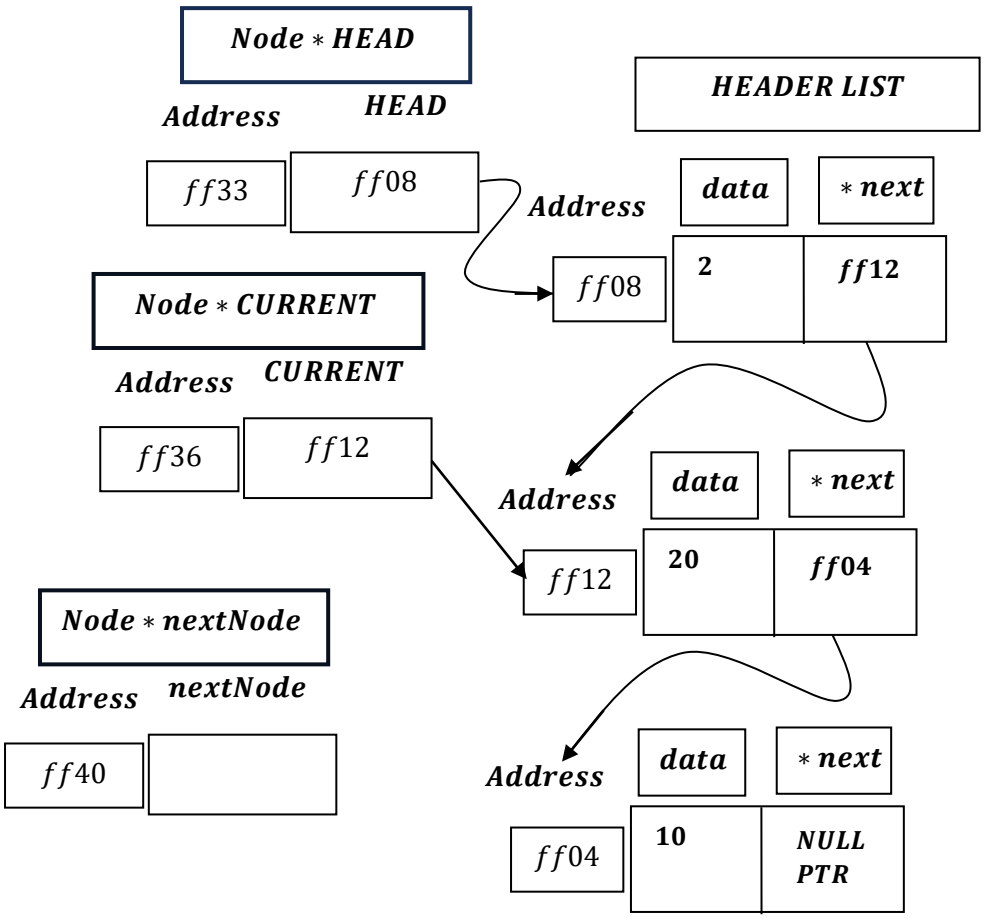
```
Node *current = head->next; // Start after header
```

$Node * CURRENT = HEAD \rightarrow NEXT = ff12.$




```
Node *nextNode;
```

*Declare Node * nextNode;*



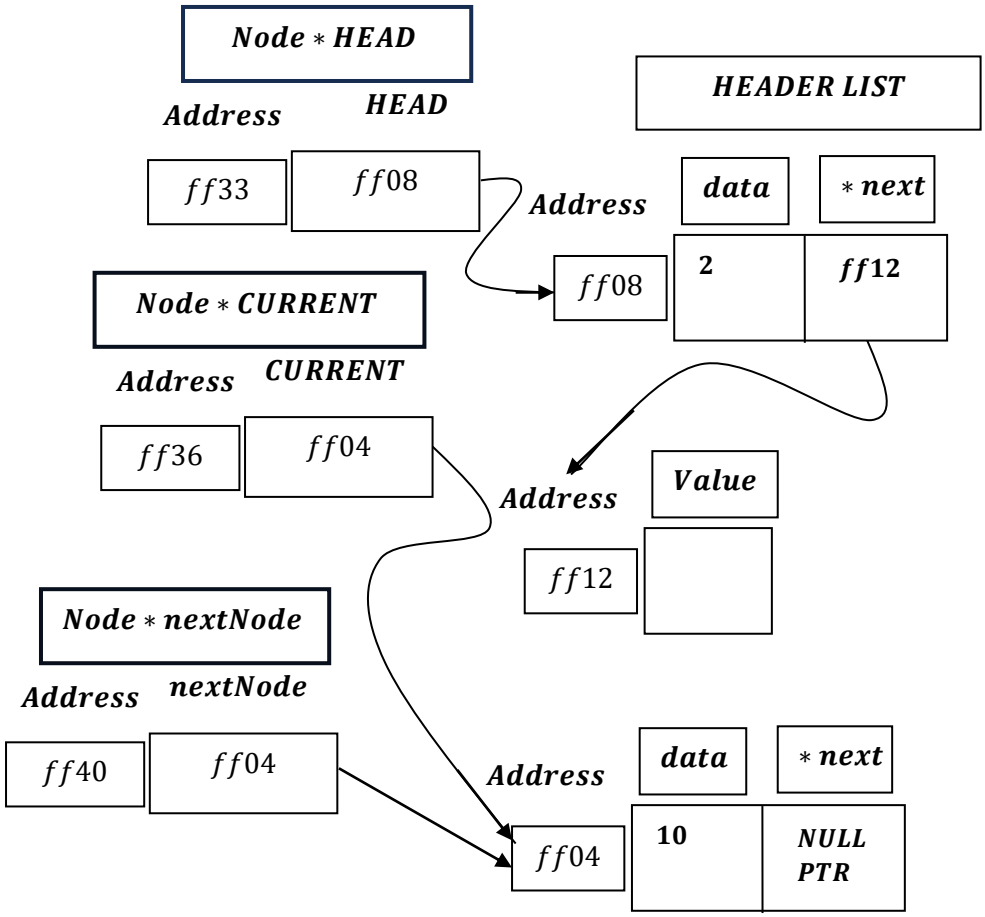

```
while (current != nullptr)
{
    nextNode = current->next;
    free(current);
    current = nextNode;
}
```

$\boldsymbol{CURRENT: ff12 \neq NULLPTR:}$

$\boldsymbol{nextNode = current \rightarrow next = ff04;}$

$\boldsymbol{free(current: ff12)}$;

$\boldsymbol{current = nextNode = ff04;}$

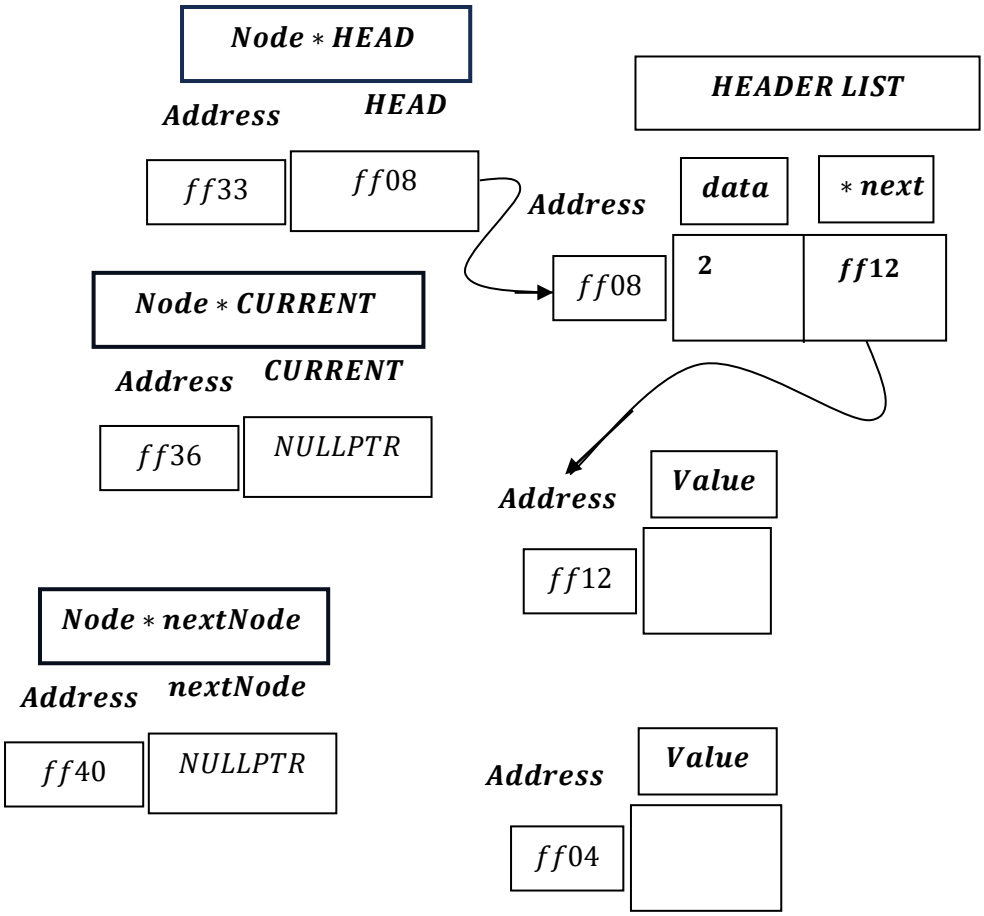


$CURRENT: ff04 \neq NULLPTR$:

$nextNode = current \rightarrow next = NULLPTR;$

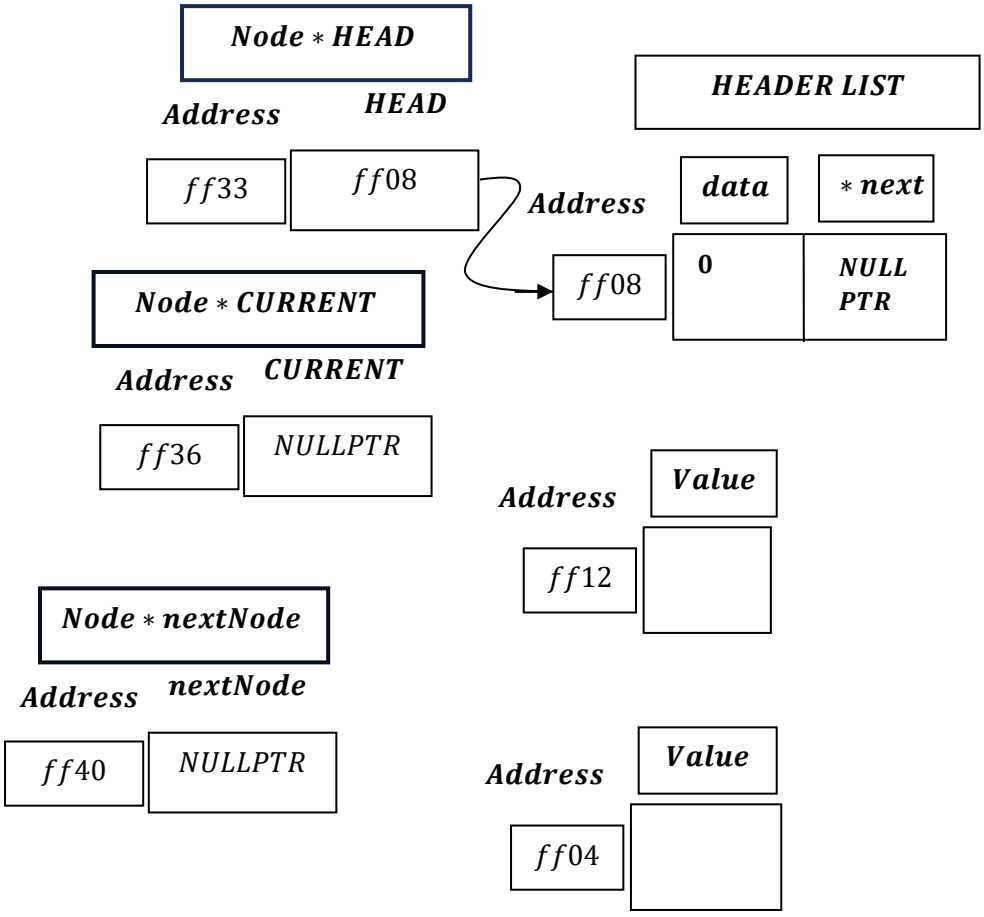
$free(current: ff04);$

$current = nextNode = NULLPTR;$



$CURRENT: NULLPTR \neq NULLPTR$ *is false and hence exit from the loop*.

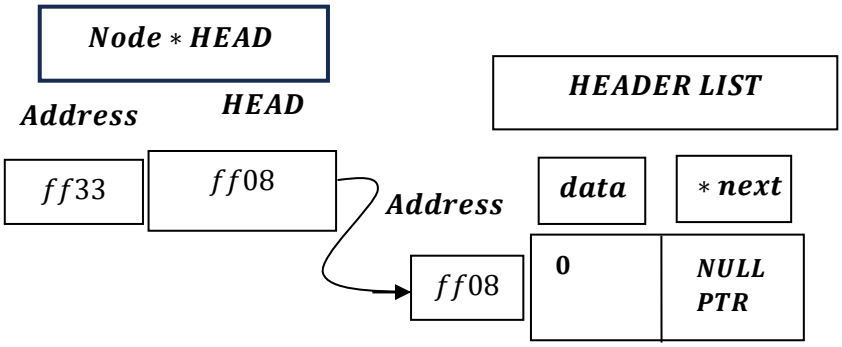
```
    head->next = nullptr;
    head->data = 0;
```




```
    cout << "List cleared (Header node preserved)." << endl;
```

***Output*: *Print* → ``*List cleared*(*Header node preserved*).``**

***Hence, after destroying the list we have* :**



Therefore, Header Node is preserved , and remain undestroyed , even if the list is fully destroyed.
